

Eksamen i BED-1304/BED-1504 Python-Lab

Fagansvarlig: Markus J. Aase

Høst 2025

Informasjon (til studenter)

Dette er eksamen i BED-1304/BED-1504, Python-lab. Den består av tre deler.

- **Del 1:** 40 flervalgsoppgaver. Ett riktig svar per oppgave. Dere får ikke minus-poeng av å svare feil.
- **Del 2:** 6 kortsvarsoppgaver som tester kodeforståelse, feilsøking og konseptforklaring. Svar kort og presist i Del 2 (maks 4–8 linjer per spørsmål).
- **Del 3:** 2 programmeringsoppgaver hvor du skal skrive Python-kode. Under eksamen, i WiseFlow, har dere ikke mulighet til å teste koden. Dette tas hensyn til i sensoreringen. Sensor er først og fremst opptatt av forståelse. Derfor anbefales på det sterkeste at dere kommenterer koden deres!

Hjelpemidler

Det er **ikke tillatt** med hjelpemidler under denne eksamenen.

Lykke til!

Del 1 - Flervalgsoppgaver

Del 1 består av flervalgsoppgaver, og det gis ett poeng per riktig svar. Det gis ikke straff ved galt svar. Av total karakter, vektes del 1 40 %.

Del 2 - Kortsvarsoppgaver

Del 2 består av 6 kortsvarsoppgaver, hvor studentene skal skrive korte tekst-svar som tester konsepter i Python, feilsøking og kodeforståelse. Poengsum per oppgave varierer, og står spesifisert lenger ned i sensorveiledningen. Av total karakter, vektes del 2 30 %.

Del 3 - Programmeringsoppgaver

Del 3 består av 2 kode-oppgaver. Det viktigste her er forståelse for det konseptuelle i Python og programmering. Poengsum per oppgave varierer, og står spesifisert lenger ned i sensorveiledningen. Av total karakter, vektes del 3 30 %.

Del 1: Flervalgsoppgaver

1. Hva skriver `print(type(12.0))` ut til konsollen?
 - a) `<class 'int'>`
 - b) `<class 'str'>`
 - c) `<class 'float'>`
 - d) `<class 'complex'>`
2. Hvilken av følgende gir eksponentiering i Python?
 - a) `^`
 - b) `**`
 - c) `sqrt()`
 - d) Både b) og c)
3. Hva er resultatet av `7 // 3` i Python?
 - a) 2.333...
 - b) 2
 - c) 3
 - d) 1
4. Hvordan definerer du en funksjon som heter `f` uten argumenter?
 - a) `function f():`
 - b) `def f():`
 - c) `fn f():`
 - d) `def f[]:`
5. Hva gjør `return` i en funksjon?
 - a) Skriver ut verdien
 - b) Avslutter funksjonen og sender tilbake en verdi
 - c) Stopper programmet
 - d) Deklarerer en variabel
6. Hvilket bibliotek er primært brukt for numeriske arrays i Python?
 - a) pandas
 - b) matplotlib
 - c) numpy
 - d) sympy
7. Hva returnerer `np.arange(1,5)`?
 - a) `array([1,2,3,4])`
 - b) `array([1,2,3,4,5])`
 - c) `array([0,1,2,3,4])`
 - d) `array([1,5])`

8. Hva gjør `np.argsort(a)` gitt en array `a`?
- a) Sorterer arrayen `a` in-place
 - b) Returnerer indeksene som sorterer `a`
 - c) Returnerer et sortert array av verdier
 - d) Gir en feilmelding hvis `a` ikke er en liste
9. Hva er den primære datastrukturen i pandas?
- a) Series
 - b) DataFrame
 - c) Panel
 - d) ndarray
10. Hvordan leser du en CSV-fil inn i en DataFrame?
- a) `pd.read_csv('fil.csv')`
 - b) `pd.load('fil.csv')`
 - c) `pd.open('fil.csv')`
 - d) `pd.DataFrame('fil.csv')`
11. Hvordan velger du kolonnen "A" fra en DataFrame `df`?
- a) `df.A`
 - b) `df['A']`
 - c) Begge metodene over
 - d) Ingen av dem
12. Hva gjør `df.describe()`?
- a) Viser deskriptiv statistikk for numeriske kolonner
 - b) Viser først 5 rader
 - c) Fjerner NaN-er
 - d) Lager grafer av kolonnene
13. Hva gjør hashtag (`#`) i Python-kode?
- a) Starter en blokk
 - b) Kommentar
 - c) Deklarerer en konstant
 - d) Marker for funksjoner
14. Hvordan bytter du verdiene til `x` og `y` uten midlertidig variabel?
- a) `x = y; y = x`
 - b) `x, y = y, x`
 - c) `swap(x,y)`
 - d) `tmp = x; x = y; y = tmp`

15. Hvilken datatype er immutable blant disse?
- a) list
 - b) dict
 - c) tuple
 - d) set
16. Hva returnerer `[1,2,3] + [4,5]`?
- a) `[1,2,3,4,5]`
 - b) `[[1,2,3],[4,5]]`
 - c) Error
 - d) `[5,7,8]`
17. Hvilket uttrykk lager en liste fra en range?
- a) `list(range(5))`
 - b) `range(5).list()`
 - c) `[range(5)]`
 - d) `range.tolist(5)`
18. Hva gjør `plt.plot(x,y)`?
- a) Lager linjeplot av y mot x
 - b) Lager linjeplot av x mot y
 - c) Lager scatterplot
 - d) Lager boksplott
19. Hvordan setter du tittel på et matplotlib-plot?
- a) `plt.header('tittel')`
 - b) `plt.title('tittel')`
 - c) `plt.set_title('tittel')`
 - d) `plt.text('tittel')`
20. Hvordan definerer du symbolske variabler x og y i sympy?
- a) `x, y = sp.var('x y')`
 - b) `x, y = sp.symbols('x y')`
 - c) `x, y = sp.math_symbol('x y')`
 - d) Både b) og c)
21. Hvilken funksjon i sympy brukes for å derivere et uttrykk?
- a) `sp.solve`
 - b) `sp.diff`
 - c) `sp.integrate`
 - d) `sp.simplify`

22. Hvilket uttrykk sjekker om `x` er lik 5 i en if-setning?

- a) `if x = 5:`
- b) `if x == 5:`
- c) `if x === 5:`
- d) `if (x equals 5):`

23. Hva gjør `df.dropna()`?

- a) Fjerner rader med minst én NaN-verdi
- b) Erstatte NaN med 0
- c) Returnerer antall NaN
- d) Lager en ny DataFrame uten endringer

24. Hvordan finner du indeks til maksimum i en Serie `s`?

- a) `s.max()`
- b) `s.argmax()`
- c) `s.idxmax()`
- d) `s.where(s==s.max())`

25. Hva vil `np.mean([1,2,3,4])` resultere i?

- a) 2
- b) 2.5
- c) 3
- d) Error

26. Hva er resultatet av `sorted([3,1,2])`?

- a) `[1,2,3]`
- b) `[3,1,2]`
- c) `{1,2,3}`
- d) `[2,3,1]`

27. Hva gjør `if x % 2 == 0:`?

- a) Sjekker om `x` er primtall
- b) Sjekker om `x` er lik 0
- c) Sjekker om `x` er partall
- d) Sjekker om `x` er oddetall

28. Hva returnerer en funksjon uten `return`-setning?

- a) 0
- b) `''`
- c) False
- d) None

29. Hvilken effekt har `np.random.seed(42)`?
- a) Gir reproduserbarhet i tilfeldige tall
 - b) Øker tilfeldighetene
 - c) Nullstiller alle variabler
 - d) Sletter arrays
30. Hvordan lager du en tom liste?
- a) `lst = {}`
 - b) `lst = []`
 - c) `lst = ()`
 - d) `lst = ''`
31. Hvilken av følgende brukes for å legge til et element bakerst i en liste?
- a) `lst.add(x)`
 - b) `lst.append(x)`
 - c) `lst.insert_end(x)`
 - d) `lst.push(x)`
32. Hvordan kan du konvertere et `float` tall, kalt `x`, til heltall (`integer`) i Python?
- a) `int(x)`
 - b) `integer(x)`
 - c) `float_to_int(x)`
 - d) `convert(int, x)`
33. Hva er hensikten til `max()` funksjonen i Python?
- a) Å finne maksimal lengde av en string
 - b) Å sammenligne to variabler
 - c) Å finne maksimum av to heltall
 - d) Å finne maksimum av en liste
34. Hva er riktig om variabler i Python?
- a) Variabelnavnet kan ikke være lenger enn 10 tegn
 - b) Du kan ikke ha mellomrom
 - c) Du må ha mellomrom
 - d) Variabelnavnet må være på norsk
35. Hva er output til følgende kode?

```
1 x = "Hello"
2 print(x[1:4])
```

- a) 'Hell'
- b) 'ello'
- c) 'ell'
- d) Error

36. Hva kan `matplotlib` brukes til?
- a) Databehandling
 - b) Visualisering av data
 - c) Numeriske beregninger
 - d) Definere funksjoner
37. Hva gjør uttrykket `len([10, 20, 30, 40])`?
- a) Returnerer summen av elementene
 - b) Returnerer antall elementer i listen
 - c) Returnerer maksimumsverdien
 - d) Returnerer siste element i listen
38. Hva vil `df['A'].mean()` gjøre i pandas?
- a) Regne ut gjennomsnittet av kolonnen "A"
 - b) Finne medianen av kolonnen "A"
 - c) Lage et histogram av kolonnen "A"
 - d) Vise de fem første radene av kolonnen "A"
39. Hvilken metode brukes for å fjerne duplikate rader i en DataFrame?
- a) `df.remove_duplicates()`
 - b) `df.drop_duplicates()`
 - c) `df.unique()`
 - d) `df.clear()`
40. Hva gjør `np.linspace(0, 1, 5)`?
- a) Lager en array med fem tilfeldige tall mellom 0 og 1
 - b) Lager en array med fem heltall fra 0 til 1
 - c) Lager en array med fem jevnt fordelte tall mellom 0 og 1
 - d) Lager en array med fem nuller

Del 2: Kortsvar

Svar kort og presist — maks 4–8 linjer per spørsmål.

1. Konsepter: Forklar forskjellen på `=` og `==` i Python. Gi et kodeeksempel.
2. Konsepter: Hva er simulering? Hvorfor gjør vi det, og hvorfor kan vi da ha bruk for `matplotlib`?
Gi to eksempler på hvor vi kan bruke simulering i Python.
3. Kodeforståelse: Hva gjør koden under? Forklar linje for linje, og forklar hva som vil printes ut.

```
1 import pandas as pd
2
3 df = pd.read_csv("data/BNP_data_2025.csv")
4 print(df.head())
5
6 df["New_Column"] = df["BNP_per_innbygger"] * df["Land_Innbyggertall"]
7
8 print(df["New_Column"])
```

4. Kodeforståelse: Hva gjør koden under? Forklar linje for linje og si hva som skrives ut.

```
1 lst = [1, 2, 3, 4, 5]
2 res = []
3 for i in lst:
4     if i % 2 == 0:
5         res.append(i**2)
6     else:
7         res.append(i*2)
8 print(res)
```

5. Feilsøking: Finn feilen(e), og skriv en korrekt versjon av funksjonen som skal avgjøre om du får frikort eller ikke basert på lønnen din. Frikortgrensen antar vi er 100 000 kr. Forklar hva som er feil, og hvorfor.

```
1 def sjekk_frikort(lønn)
2 if lønn < 100000:
3     resultat = Du får frikort
4 else:
5     resultat = Du må betale skatt
6     return resultat
```

6. Kodeforståelse: Her bruker vi biblioteket SymPy, forklar linje-for-linje hva som skjer her.

```
1 import sympy as sp
2 x = sp.symbols('x')
3 f = x**3 - 3*x**2 + 2
4 df = sp.diff(f, x)
5 pnkt = sp.solve(sp.Eq(df, 0), x)
6 print(pnkt)
```

Del 3: Programmeringsoppgaver

I denne oppgaven skal dere skrive Python kode, etter beste evne. Kommenter koden deres godt.

Du har ikke mulighet til å teste koden, og vi vil derfor se forbi noen små syntaks-feil. Likevel, skal du (etter beste evne) skrive kode som kan kjøres i Python.

Lykke til!

Oppgave 1 - Bankrådgiver

Du arbeider som rådgiver i en bank, og skal hjelpe folk med å sette opp sparemål. Det sparemålet kan være til ferie, egenkapital til bolig, turutstyr eller hva enn kundene dine ønsker å spare til.

Du hjelper kundene dine én og én, og skriver ny kode hver eneste gang. Så får du en idé, og tenker at dette kan du løse på en smartere måte!

Skriv et Python-program som gjør følgende:

- Lag en funksjon `sparing()` som tar inn følgende parametere: `antatt_rente`, `startkapital` og `sparemaal`.
- Funksjonen skal bruke en `while`-løkke for å finne ut hvor lang tid det tar for kunden å nå sparemålet sitt.
- Funksjonen skal returnere hvor mange år de må spare for å nå måle sitt. Funksjonen skal også returnere `antatt_rente`, `startkapital`.
- Til slutt, skal du skrive en `print()`-setning som formidler denne informasjonen. Du kan anta følgende eksempelverdier:

- `antatt_rente = 0.05` (5 %)
- `startkapital = 10 000` kr
- `sparemaal = 20 000` kr

Husk å kommenter koden!

Oppgave 2 — Salgsdata

Du jobber som analytiker for en bedrift som driver med salg av ulike varer. Du har brukt dine Python-ferdigheter til å laste inn en dataramme, kalt `df`, som består av pris og antall solgte enheter av ulike produkter. Det gjør du ved koden under:

```
1 # Importerer Pandas
2 import pandas as pd
3
4 df = pd.read_csv('data/salgsdata.csv')
5 df.head()
```

Da ser dataen din slik ut, med kolonnene 'Vare', 'Pris' og 'Antall_varer_solgt'.

	Vare	Pris	Antall_varer_solgt
0	Jakke	1200	5
1	Sko	800	8
2	Sekk	650	4
3	Lue	250	12
4	Caps	350	16

Bedriften du jobber i driver med salg av vare i både Norge, og i andre EU-land. 'Pris' kolonnen er oppgitt i norske kroner.

Sjefen din lurer på om du kan fikse følgende:

1. Lag en ny kolonne som heter 'Pris_EU', som gir prisen av salgsvarene i Euro. Anta at 1 € = 11.63 NOK.
2. Bruk en for-løkke for å itere gjennom datarammen `df`, og lag en liste som inneholder de varene som koster mer enn 700 kroner.

Skriv et Python program som gjør oppgavene over.

HINT:

Under er en oversikt over noen potensielt nyttige funksjoner i Python:

Funksjon / metode	Forklaring
<code>len(df)</code>	Returnerer antall rader i en DataFrame. Brukes ofte i en <code>for</code> -løkke for å iterere gjennom alle radene.
<code>df.loc[i, 'Kolonne']</code>	Henter verdien i rad <code>i</code> og kolonne 'Kolonne' (kolonne-navnet). Kan også brukes til å endre verdier.
<code>append()</code>	Legger til et nytt element i en liste.
<code>range(n)</code>	Lager en sekvens av tall fra 0 til (n-1). Kan ofte brukes sammen med <code>len()</code> .
<code>df['Kolonne'] = ...</code>	Lager en ny kolonne eller endrer en eksisterende kolonne i DataFrame.
<code>sum(liste)</code>	Returnerer summen av alle elementene i en liste (kun tall).
<code>print()</code>	Skriver ut tekst eller variabler til konsollen.
<code>type(x)</code>	Viser hvilken datatype et objekt har (for eksempel <code>int</code> , <code>str</code> , <code>float</code>).
<code>round(x, 2)</code>	Runder av tallet <code>x</code> til 2 desimaler.
<code>df.head()</code>	Viser de første 5 radene i en DataFrame (nyttig for å sjekke data).

Løsningsforslag

Del 1 — Flervalgsoppgaver (sensorveiledning)

Her listes riktige svar (bokstav) for hver oppgave i Del 1 i samme rekkefølge som oppgavene.

1. c
2. b
3. b
4. b
5. b
6. c
7. a
8. b
9. b
10. a
11. c
12. a
13. b
14. b
15. c
16. a
17. a
18. a
19. b
20. b
21. b
22. b
23. a
24. c
25. b
26. a
27. c
28. d

- 29. a
- 30. b
- 31. b
- 32. a
- 33. d
- 34. b
- 35. c
- 36. b
- 37. b
- 38. a
- 39. b
- 40. c

Del 2 — Kortsvar (løsningsforslag)

1. **Forskjellen på = og ==:** = er tilordningsoperator (setter en verdi til en variabel). == er en sammenligningsoperator som sjekker likhet. Eksempel:

```
1 x = 5      # = tildeler 5 til x
2 if x == 5:
3     print("x er lik 5")
4
```

2. **Hva er en simulering?** *Simulering* er noe vi bruker når vi ønsker å replikere et fenomen fra den ekte verden. Det vil ofte kreve tilfeldigheter, og utvikling over tid. Ofte er vi interessert i svaret i dag, om hvordan en investering kan se ut om 10 år. Da kan vi bruke `numpy` sin `random` funksjon for å ta med tilfeldigheter i simuleringen. `matplotlib` er ofte nyttig i simuleringsoppgaver, da det kan gi en visuell forståelse av prosessen.

3. Kodeforståelse:

- **Linje 1: `import pandas as pd`**
Importerer `pandas`-biblioteket, som er standardverktøyet i Python for dataanalyse og databehandling. Vi gir det aliaset (kallenavnet) `pd`.
- **Linje 3: `df = pd.read_csv(data/BNP_data_2025.csv)`**
Denne linjen bruker `pandas` til å lese inn data fra en CSV-fil som ligger i mappen `data` og heter `BNP_data_2025.csv`. Dataene lastes inn i en variabel kalt `df`, som nå er en `DataFrame` (en tabell).
- **Linje 4: `print(df.head())`**
Denne linjen ber om å få vist de første 5 radene av `DataFrame`-en `df`. Dette er en kommando som ofte brukes i Python/Pandas for å inspisere hvordan dataene ser ut.
- **Linje 6: `df['New_Column'] = df[BNP_per_innbygger"] * df[Land_Innbyggertall"]`**
Her oppretter vi en helt ny kolonne i `DataFrame`-en vår som heter `New_Column`. Verdien for hver rad i denne nye kolonnen blir beregnet ved å ta verdien fra `BNP_per_innbygger`-kolonnen og multiplisere den med verdien fra `Land_Innbyggertall`-kolonnen. Denne operasjonen regner i praksis ut det totale BNP-et for hver enhet (f.eks. land eller region) i datasettet.
- **Linje 8: `df['New_Column']`**
Denne linjen henter ut den nye kolonnen vi nettopp laget.
- **Hva skrives ut?**
Den siste linja `print(df['New_Column'])` skrive ut innholdet i denne kolonnen, altså en liste over det totale BNP-et for alle radene i datasettet.

4. Kodeforståelse (hva gjør koden?):

```
1 lst = [1, 2, 3, 4, 5]
2 res = []
3 for i in lst:
4     if i % 2 == 0:
5         res.append(i**2)
6     else:
7         res.append(i*2)
8 print(res)
9
```

Forklaring linje for linje:

- Lager listen `lst` og en tom liste `res`.
- Itererer gjennom hvert element `i` i `lst`.
- Hvis `i` er partall (delelig med 2) legges `i**2` (kvadratet) inn i `res`.
- Ellers legges `i*2` (dobbel verdi) inn i `res`.
- Til slutt skrives `res` ut. Output blir: `[2, 4, 6, 16, 10]`.

5. Feilsøking — frikort-funksjon: Feil i originalkoden:

- Mangler kolon (`:`) etter funksjonsdefinisjonen
- `if` og `else` skal ha innrykk.
- `return` må komme riktig plass. Ikke "inne i" `else`, slik oppgaven viser.
- Teksten mangler anførselstegn for å definere det som `str` (en streng).

Korrekt versjon:

```

1 def sjekk_frikort(lønn):
2     if lønn < 100000:
3         resultat = "Du får frikort"
4     else:
5         resultat = "Du må betale skatt"
6     return resultat
7

```

6. SymPy — kritiske punkter (syntaks):

- **Linje 1: `import sympy as sp`**
 Dette importerer SymPy-biblioteket, som er et Python-bibliotek for symbolsk matematikk (algebra, derivasjon, etc.). Vi gir det aliaset `sp` for å kunne skrive kortere kode.
- **Linje 2: `x = sp.symbols('x')`**
 Her definerer vi `x` som et symbolsk variabel. Dette er ikke en vanlig variabel med en tallverdi, men et matematisk symbol x som SymPy kan utføre algebraiske operasjoner på.
- **Linje 3: `f = x**3 - 3*x**2 + 2`**
 Vi definerer en funksjon $f(x)$ ved å bruke det symbolske `x`-objektet. `f` er nå et symbolsk uttrykk som representerer polynomet $f(x) = x^3 - 3x^2 + 2$.
- **Linje 4: `df = sp.diff(f, x)`**
 Dette er den symbolske derivasjonen. Vi bruker `sp.diff()`-funksjonen til å derivere uttrykket `f` med hensyn på symbolet `x`. Resultatet `df` vil være det nye symbolske uttrykket $f'(x) = 3x^2 - 6x$.
- **Linje 5: `pnkt = sp.solve(sp.Eq(df, 0), x)`**
 Denne linjen løser ligningen $f'(x) = 0$ for å finne de stasjonære punktene.
 - `sp.Eq(df, 0)` lager en formell ligning (`Equation`) der den deriverte `df` settes lik 0.
 - `sp.solve(...)` løser denne ligningen for `x`.
 - Resultatet, en liste med løsninger, lagres i variabelen `pnkt` (punkter).
- **Linje 6: `print(pnkt)`**
 Dette skriver ut listen med løsninger som ble funnet i forrige linje. Siden ligningen er $3x^2 - 6x = 0$, som kan faktorerises til $3x(x - 2) = 0$, vil løsningene være $x = 0$ og $x = 2$. Utskriften vil være `[0, 2]`.

Del 3 — Programmeringsoppgaver (løsningsforslag)

Oppgave 1 — Bankrådgiver

Nedenfor er et løsningsforslag for hvordan man kan løse oppgaven, men ikke den eneste.

```
1 # Definerer funksjonen med de oppgitte parameterne
2 def sparing(antatt_rente, startkapital, sparemål):
3     # Initialiserer kapital med startkapitalen, og variabelen 'år' med 0
4     kapital = startkapital
5     år = 0
6
7     # While-løkke for å beregne kapitalvekst over tid
8     while kapital < sparemål:
9         kapital += kapital * antatt_rente # Økende kapital
10        år += 1                          # Øker antall år
11
12    # Returnerer resultatene
13    return år, antatt_rente, startkapital
14
15 # Kaller funksjonen med spesifikke verdier
16 år, antatt_rente, startkapital = sparing(0.05, 10000, 20000)
17 # Printer resultatet
18 print(f"\nDu når sparemålet ditt på 20000 kr etter {år} år, med startkapital på
      {startkapital:.2f} kr og rente på {antatt_rente * 100}%.")
```

Oppgave 2 — Salgsdata

Vi benytter flere av funksjonene som er foreslått i oppgavebeskrivelsen.

- `df['Pris_EU'] = ...` bruker vi for å beregne prisen i Euro. Her kan man også bruke `round()` funksjonen for å runde av svaret man får.
- Bruker en `for`-løkke for å iterere gjennom datarammen vår.
- Bruker `df.loc()` og `append()` for å hente hvilke varer som er dyrere enn 700 kr.
- Dette skal kommenteres godt.

```
1 # Del 1
2 import pandas as pd
3
4 # Lager DataFrame
5 data = {
6     'Vare': ['Jakke', 'Sko', 'Sekk', 'Lue', 'Caps'],
7     'Pris': [1200, 800, 650, 250, 350],
8     'Antall_varer_solgt': [5, 8, 4, 12, 16]
9 }
10
11 df = pd.DataFrame(data)
12
13 # Setter valutakurse, og lager en ny kolonne med Euro prisen
14 valutakurs = 11.63
15 df['Pris_EU'] = df['Pris'] / valutakurs
16
17 # Del 2
18 # Lager en tom liste for varer over 700 kr
19 varer_over_700 = []
20
21 # Løkke for å finne varer med pris over 700 kr
22 for i in range(len(df)):
23     if df.loc[i, 'Pris'] > 700:
24         varer_over_700.append(df.loc[i, 'Vare']) # df.loc brukes for å iterere
25         gjennom de ulike varene, og sjekker hvilke som er dyrere enn 700 kr
26
27 # printer varene som er dyrere enn 700 kr
28 print(df)
29 print("Varer som koster mer enn 700 kr:", varer_over_700)
```